PUCRS ESCOLA POLITÉCNICA

98800-04

FUNDAMENTOS DE SISTEMAS COMPUTACIONAIS

Sistemas de Numeração

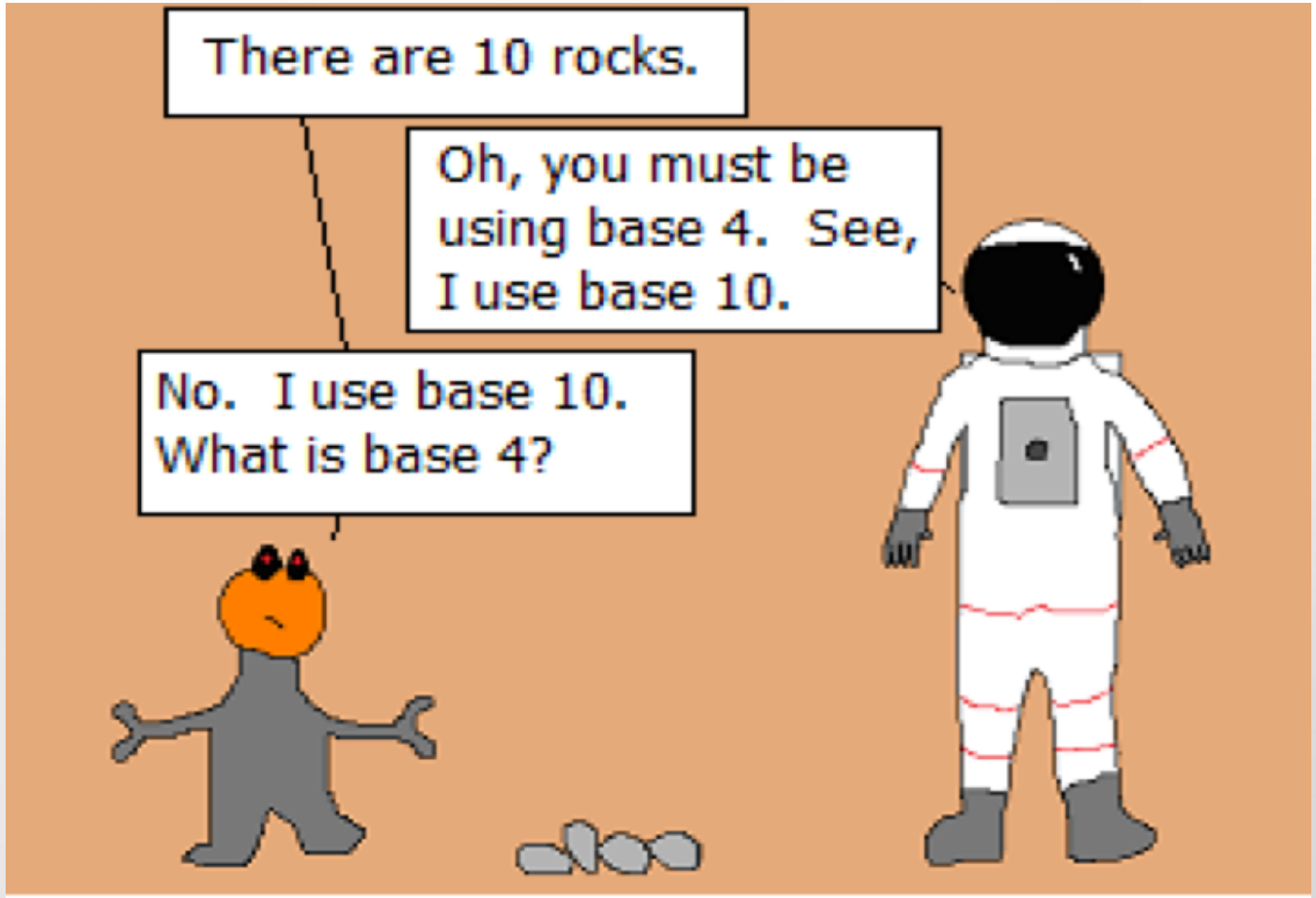
Prof. Iaçanã Ianiski Weber

MARISTA

Baseado no material gentilmente cedido pelo ***Prof. Dr. Sérgio Johann Filho***

Toda a base é Base 10...?

Quantas pedras?



Every base is base 10.

Binário, Decimal, Hexadecimal

Número vs Numeral

Numeral

Um símbolo ou nome que representa um número

por exemplo: 4, four, quatro, IV, IIII, …

…e **dígitos** são símbolos que formam numerais

Acima da linha de abstração

Abstraction Line

Abaixo da linha de abstração

Número

A "ideia" em nossas mentes... existe apenas UMA dessas:

o conceito de "4"

Sistemas de Numeração

- Um Sistema de Numeração é um conjunto de símbolos utilizados para **representação de quantidades** e **as regras que definem a forma de representação**.
  - É determinado fundamentalmente pela Base (número de símbolos utilizados).

- Usualmente, tem-se dois tipos de sistemas:
  - Sistemas Posicionais e Não-Posicionais.

Sistemas de Numeração

Sistemas Não-Posicionais:

- São aqueles em que o valor atribuído a um símbolo não se altera, independentemente da posição em que ele se encontra no conjunto de símbolos que está representando uma quantidade.
- Ex.: ***sistema romano***
  - I = 1, V = 5, X = 10, L = 50, C = 100, D = 500, M = 1000

X X I - 10 10 1 (10+10+1=21)
X I X - 10 1 10 (10+10-1=19)

Arabic numerals				
0				
1	M	C	X	I
2	MM	CC	XX	II
3	MMM	CCC	XXX	III
4		CD	XL	IV
5		D	L	V
6		DC	LX	VI
7		DCC	LXX	VII
8		DCCC	LXXX	VIII
9		CM	XC	IX

Sistemas Posicionais:

- São aqueles em que o valor atribuído a um símbolo depende da posição em que ele se encontra no conjunto de símbolos que está representando uma quantidade.
- Ex.: ***sistema decimal*** (símbolos 0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
- 343
- unidade
- dezena
- centena

300 + 40 + 3 = 343

Sistemas de Numeração

- **Base**: conjunto de símbolos possíveis a serem utilizados, representado por “b”
- **Princípio**: cada posição em um número vale “b” vezes o que vale a posição imediatamente posterior à direita.
- Um número "x" qualquer numa base "b" é escrito na forma polinomial

$$x = (a_n a_{n-1} \ldots a_1 a_0)_b$$

- Ou seja:

$$x = (a_n b^n + a_{n-1} b^{n-1} + \ldots + a_1 b^1 + a_0 b^0)$$

$$x = \sum_{i=0}^{n} (a_i b^i)$$

Decimal: Base 10 (dez)

Dígitos: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Exemplo:

$\mathbf{3271} = \mathbf{3271}_{10} = (\mathbf{3} \times 10^3) + (\mathbf{2} \times 10^2) + (\mathbf{7} \times 10^1) + (\mathbf{1} \times 10^0)$

Binário: Base 2 (dois)

Dígitos: 0, 1 (***bi**nary digi**ts*** -> **bits**)

Exemplo: Número Binário "**1101**"

Converter para decimal:

0b1101 = $\mathbf{1101}_2$ = $(1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$

= 8 + 4 + 0 + 1

= 13

Hexadecimal: Base 16 (dezesseis)

Dígitos: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

10, 11, 12, 13, 14, 15

Exemplo: Número hexadecimal "**A5**"

Converter para decimal:

0xA5 = $\mathbf{A5}_{16}$ = $(\mathbf{10} \times 16^1) + (\mathbf{5} \times 16^0)$

= 160 + 5

= 165

Um número binário com *n*-bits pode acomodar ***2^n valores***.

Sistemas de Numeração

- **Exemplos de representação polinomial**:
  - Sistema Decimal:
  - $(343)_{10}$
    - valor =
  - Sistema Binário:
  - $(1011)_2$
    - valor =
  - Sistema Octal:
  - $(127)_8$
    - valor =
  - Sistema Hexadecimal:
  - $(3F)_{16}$
    - valor =

- **Exemplos de representação polinomial**:
  - Sistema Decimal:
  - $(343)_{10}$
    - valor = $3.10^2 + 4.10^1 + 3.10^0 = 300 + 40 + 3 = 343$
  - Sistema Binário:
  - $(1011)_2$
    - valor = $1.2^3 + 0.2^2 + 1.2^1 + 1.\ 2^0 = 8 + 2 + 1 = 11$
  - Sistema Octal:
  - $(127)_8$
    - valor = $1.8^2 + 2.8^1 + 7.8^0 = 64 + 16 + 7 = 87$
  - Sistema Hexadecimal:
  - $(3F)_{16}$
    - valor = $3.16^1 + F.16^0 = 3.16^1 + 15.16^0 = 48 + 15 = 63$

Conversão BIN → DEC

Colocar o número na forma polinomial e resolver o polinômio

Exemplo: 110001_2

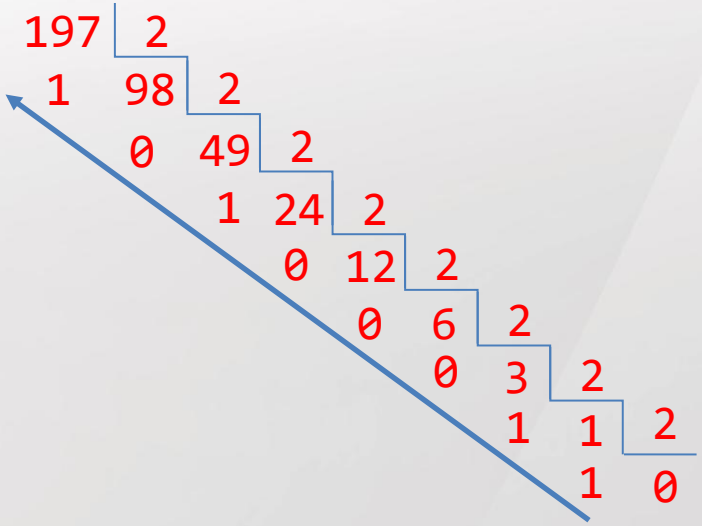
$$= 1.2^5+1.2^4+0.2^3+0.2^2+0.2^1+1.2^0$$

$$= (49)_{10}$$

Conversão DEC → BIN

Através de divisões sucessivas do número a ser transformado por 2 até obter o quociente zero. Após, toma-se os restos na ordem inversa a que foram obtidos.

Exemplo: 197_{10}



$= (11000101)_2$

Nibbles e Bytes

- 4 Bits
  - 1 "**Nibble**"
  - 1 Hex Digit = 16 coisas
- 8 Bits
  - 1 "**Byte**"
  - 2 Hex Digits = 256 coisas

Dec	Hex	Bin
00	0	0000
01	1	0001
02	2	0010
03	3	0011
04	4	0100
05	5	0101
06	6	0110
07	7	0111
08	8	1000
09	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Memorize essa tabela!

Converter Binário <-> Hexadecimal

- Binário -> Hex? **Fácil**!
  - Agrupar em nibbles (4 bits)
  - Se necessário, preencha com 0's à esquerda
  - `0b11110`
    → `0b00011110`
    (→ `0b0001 1110`)
    → **`0x 1 E`**

- Hex -> Binário? **Fácil**!
  - Basta substituir cada dígito por um nibble!
  - `0x2A`
    → `0b00101010`
    → **`0b101010`** (remova os zeros à esquerda)

Dec	Hex	Bin
00	0	0000
01	1	0001
02	2	0010
03	3	0011
04	4	0100
05	5	0101
06	6	0110
07	7	0111
08	8	1000
09	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Memorize essa tabela!

Sistema Hexadecimal

- Utilizado como abreviação para números binários.
  - Sequências longas de bits são difíceis de ler e transcrever.
- Números hexadecimais são chamados de base 16.
- Símbolos: `0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F`

Conversão BIN → HEX

- Separa-se o número, da direita para a esquerda, em grupos de 4 bits (*nibble*).
- Se o último grupo (mais significativo) tiver menos de 4 bits, completar com zeros.
- Substitui-se cada grupo pelos respectivos símbolos hexadecimais.

$(1111\ 0001)_2 =$

$(10\ 1011)_2 =$

Conversão BIN → HEX

- Separa-se o número, da direita para a esquerda, em grupos de 4 bits (*nibble*).
- Se o último grupo (mais significativo) tiver menos de 4 bits, completar com zeros.
- Substitui-se cada grupo pelos respectivos símbolos hexadecimais.

$(1111\ 0001)_2 = (F1)_{16}$

$(10\ 1011)_2 = (0010\ 1011)_2 = (2B)_{16}$

Conversão HEX → BIN

- Substituir cada dígito hexadecimal por seu correspondente grupo de 4 bits.

$(AF)_{16}$ =

Conversão HEX → BIN

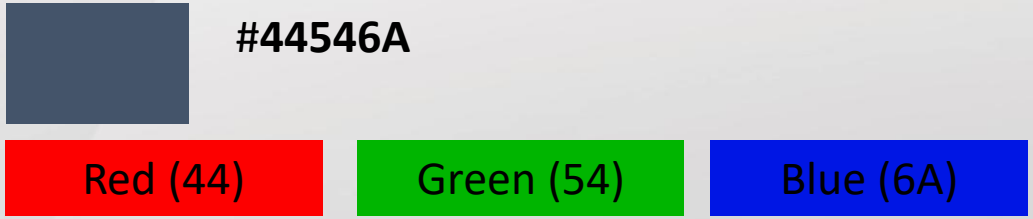
- Substituir cada dígito hexadecimal por seu correspondente grupo de 4 bits.

$(AF)_{16} = (1010\ 1111)_2$

BIG IDEA: Bits pode representar qualquer coisa!!

- Valores lógicos? 1 bit
  - Uma possibilidade é: 0 → False, 1 → True

- Caracteres? Várias opções:
  - A, …, Z: 26 letras → 5 bits ($2^5 = 32$)
  - ASCII: maiúsculo/minúsculo + pontuação → 7 bits → arrendondado para 1 byte
  - Unicode (www.unicode.com): código padrão que cobre todas as línguas do planeta ⇒ 8, 16, 32 bits



- Cores?
  - Código de cores HTML: 24 bits (3 bytes)
- Lacalização / endereços?
  IPv4 (32 bit), IPv6 (64 bit), etc.

Com N bits, você consegue distinguir até 2^N coisas.

Exercícios

1 - Apresente os cálculos completos e os valores referentes às seguintes conversões entre bases **binária** (inteiros não negativos), **decimal** e **hexadecimal**:

$$101101_2 = (?)_{10}$$

$$= 2^5 + 2^3 + 2^2 + 2^0$$

$$= 32+8+4+1$$

$$= (45)_{10}$$

Exercícios

1 - Apresente os cálculos completos e os valores referentes às seguintes conversões entre bases **binária** (inteiros não negativos), **decimal** e **hexadecimal**:

$110110_2 = (?)_{16}$

$= 0011\ 0110$

$= \quad 3 \quad\ \ 6$

$= (36)_{16}$

Exercícios

1 - Apresente os cálculos completos e os valores referentes às seguintes conversões entre bases **binária** (inteiros não negativos), **decimal** e **hexadecimal**:

$$37_{10} = (?)_2$$

> **10 0101**

37	2				
1	18	2			
	0	9	2		
		1	4	2	
			0	2	2
				0	**1**	**2**
					1	**0**

Exercícios

1 - Apresente os cálculos completos e os valores referentes às seguintes conversões entre bases **binária** (inteiros não negativos), **decimal** e **hexadecimal**:

$$110_{10} = (?)_{16}$$

Opção 1:

Primeiro passo:

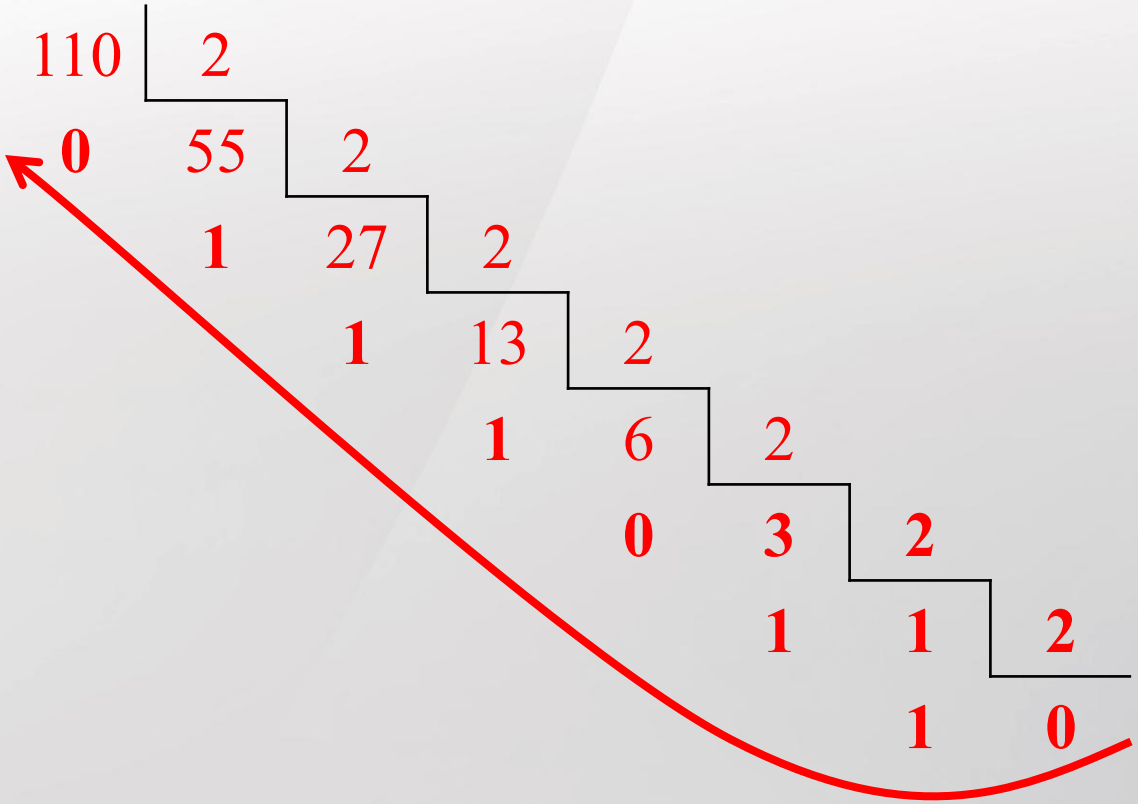
DEC -> BIN

- **0110 1110**

Segundo passo:

BIN -> HEX

- **$6E_{16}$**



Exercícios

1 - Apresente os cálculos completos e os valores referentes às seguintes conversões entre bases **binária** (inteiros não negativos), **decimal** e **hexadecimal**:

$$110_{10} = (?)_{16}$$

Opção 2: $6E_{16}$

110	16	
14	6	16
	6	0

Exercícios

1 - Apresente os cálculos completos e os valores referentes às seguintes conversões entre bases **binária** (inteiros não negativos), **decimal** e **hexadecimal**:

$$\texttt{cb03}_{16} = (?)_{10}$$

1100 1011 0000 0011

$= 2^{15} + 2^{14} + 2^{11} + 2^{9} + 2^{8} + 2^{1} + 2^{0}$

$= 32768+16384+2048+512+256+2+1$

$= (51971)_{10}$

Exercícios

1 - Apresente os cálculos completos e os valores referentes às seguintes conversões entre bases **binária** (inteiros não negativos), **decimal** e **hexadecimal**:

$$51_{16} = (?)_2$$

$$\mathbf{(0101\ 0001)_2}$$

Como representar um número inteiro?

- Queremos uma representação que comporte as operações comuns:
  - Adição
  - Subtração
  - Multiplicação
  - Divisão
  - Comparação (<, =, ≠, ≤, etc.)

- Exemplo: 10 + 7 = 17
  - 10, 7 pode ser representado com 4 bits:
  - Soma e subtração podem ser feitas da mesma forma que fazemos em decimal!!
  - Mas aqui temos um problema…

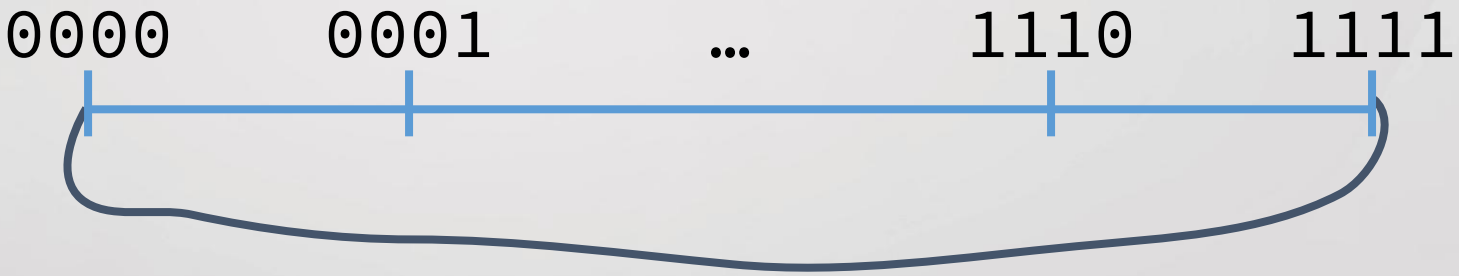
```
  11     carry bits
  1010
+ 0111
-------
 10001
```

E se o número for "muito grande"? *Overflow*

- Estritamente falando, os numerais de base 2 têm um número ∞ de dígitos.
  - Nós só não representamos esses dígitos

…00000001010

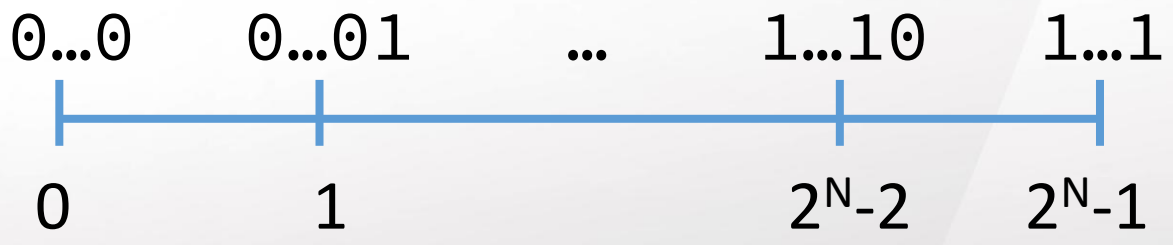
- Contudo, **o hardware tem limites físicos**. Sem bits infinitos!
  - Representações comuns: 8 bits, 16 bits, 32 bits, 64 bits, ...
  - Novamente: Com N bits, você pode representar no máximo 2^N coisas.
- Se o resultado inteiro da operação (+, -, *, /, >, <, =, etc.) não pode ser representado pelos bits do HW, dizemos que ocorreu ***Overflow***



Overflow **(estouro de representação): o resultado aritmético está fora do intervalo representável.**

Muitas representações numéricas possíveis

- Até agora, discutimos apenas ***unsigned numbers*** (não-negativos).
  - A maioria dos computadores usa a representação "óbvia":

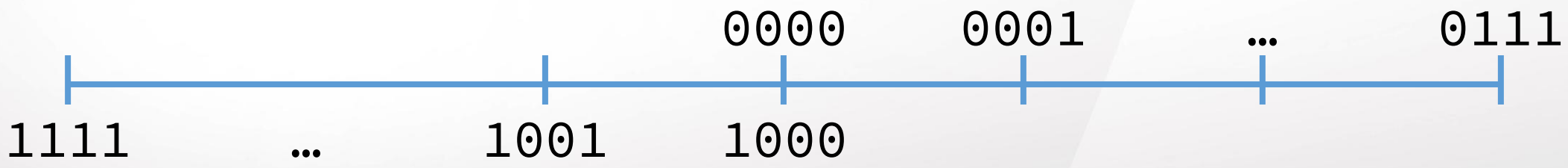


- E os ***signed numbers***? Precisamos de uma maneira de representar números negativos. Vamos discutir algumas:
  - Sinal-Magnitude
  - Complemento
  - Complemento de dois (***única representação inteira com sinal adotada nessa disciplina***)

Sinal Magnitude e Complemento

Sinal-Magnitude: Não há almoço grátis

- A solução "obvia" :
  - O **bit mais a esquerda define o sinal**: 0 → +, 1 → – O restante define o valor numérico



- **Sinal-Magnitude** é **raramente utilizado**, devido a muitas deficiências:
  - "Incrementar" aumenta os valores às vezes, diminui os valores outras vezes
  - Circuito aritmético complicado: etapas especiais dependendo se os sinais são iguais/diferentes
  - **Dois zeros** (Como comparar??)
- Razoável para processamento de sinal, não para computadores de uso geral

0x00000000 = $+0_{10}$
0x80000000 = -0_{10}

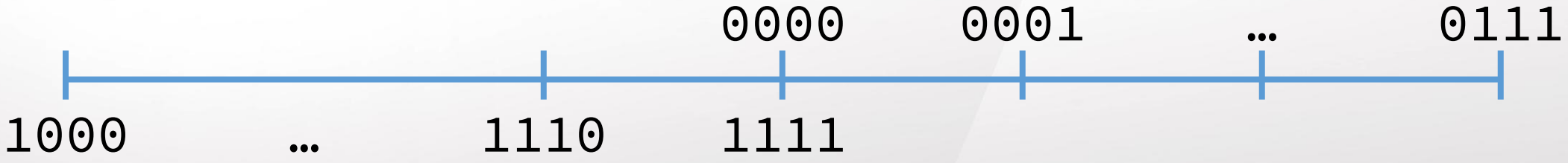
0b 1000 0000 … 0000

Complemento de um: outra tentativa

- Para representar um número negativo, complemente ("**inverta**") os bits de sua representação positiva:

$+7_{10}$ = 0b 0000 0111

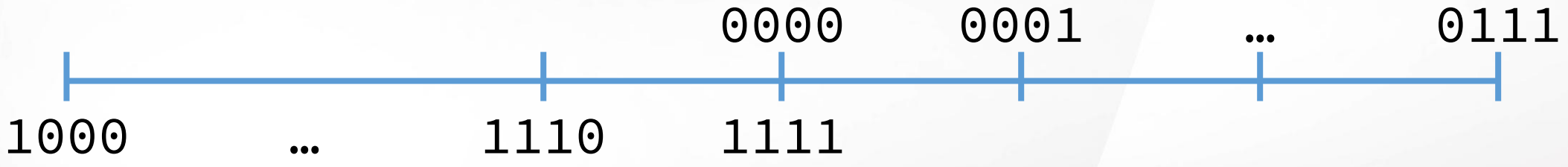
-7_{10} = 0b 1111 1000



- Observações:
  - Números positivos: 0s à esquerda
  - Números negativos: 1s à esquerda

- #s representado em N bits:
  - Zero: 2
  - Positivo: $(2^N)/2 - 2$
  - Negativo: (same as positive)

Deficiências do complemento?

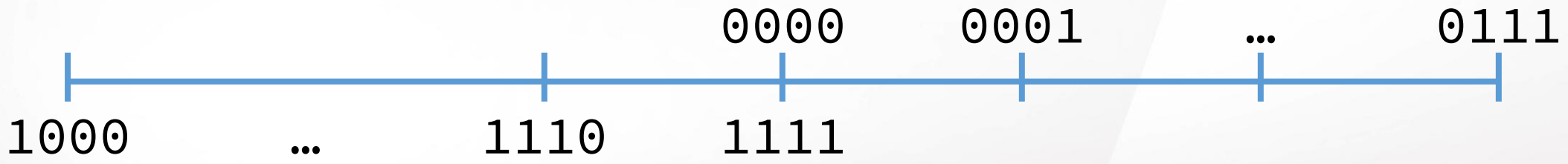


- Vantagens:
  - Bit mais à esquerda ("**bit mais significativo**") ainda é efetivamente um bit de sinal
  - "Incrementar" em binário é consistente na linha de representação
- Algumas desvantagens ainda persistem:
  - Ainda dois zeros
  - Aritmética ainda um pouco complicada

Complemento de Dois

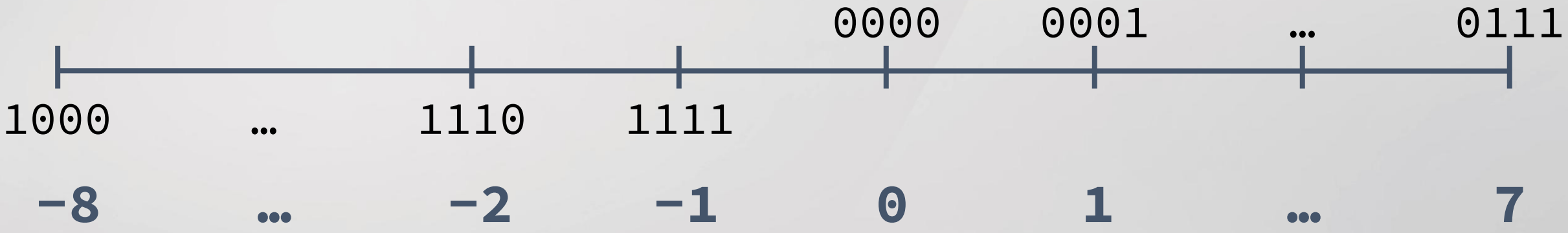
Complemento de Dois

- Complemento de um:

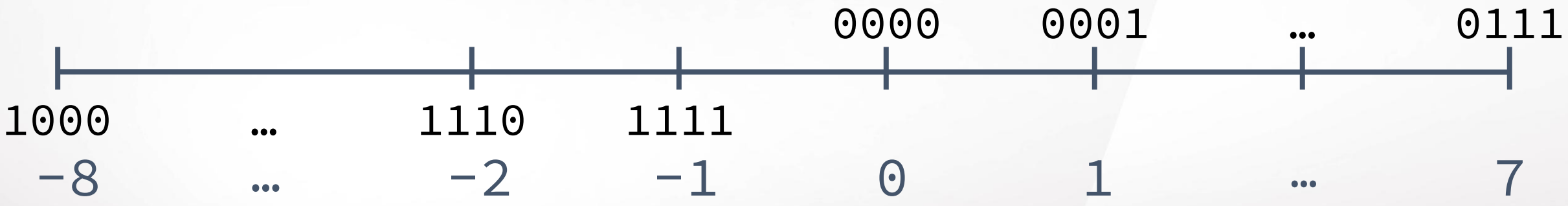


- O problema: os negativos "se sobrepõem" aos positivos, criando os dois 0s.

- A solução: **Deslocar números negativos em um**.



Aritmética é simples



- Vantagens:
  - Bit mais à esquerda ("**bit mais significativo**") ainda indica o sinal
  - Incrementar em binário é consistente com a linha de representação
  - Agora temos apenas um "zero"
  - **Hardware simples para adição**

Decimal

```
  5
+ -5
  0
```

Complemento de dois

```
  111   carry bits
  0101
+ 1011
dropped 10000
```

Complemento de dois: conversão

- Números positivos **e** negativos podem ser calculados usando a mesma fórmula:
- Valores de bit multiplicados por potências de 2!!
- O hardware para converter positivo em negativo (e vice-versa) é **simples**.

1. Complemente todos os bits
2. Em seguida, adicione 1

0b1011

$= (1 \times -2^3) + (0 \times 2^2) + (1 \times 2^1) + (1 \times 2^0)$

$= -8 + 0 + 2 + 1$

$= -5$

0b0101

$= (0 \times -2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$

$= 0 + 4 + 0 + 1$

$= 5$

5_{10} 0101 → 1010 → 1011 -5_{10}

-5_{10} 1011 → 0100 → 0101 5_{10}

Complemento de Dois

- **O complemento de dois é a <u>representação numérica padrão</u> para inteiros com sinal.**

  - 2^{N-1} negativos
  - 2^{N-1} não-negativos
    - 1 zero
    - Quantos positivos?

E em resumo...

- Representamos "coisas" em computadores como padrões de bits específicos:
  - Com N bits, você pode representar no máximo 2^N coisas.
- Hoje, discutimos codificações diferentes para números inteiros:
  - Inteiros sem sinal
  - Inteiros com sinal:
    - Sinal-Magnitude
    - Complemento
    - Complemento de Dois
- Arquitetos de computador tomam decisões de design para simplificar o hardware
  - **O complemento de dois é o padrão.**

E em resumo...

***Overflow:** É quando o resultado de uma operação aritmética está fora do intervalo representável de inteiros.*

- Os números têm dígitos infinitos, mas os computadores têm precisão finita. Isso pode levar a erros aritméticos.
- **Atenção!** *Overflow* é identificado de formas diferentes quando estamos trabalhando com *unsigned* e *signed.*

Unsigned

```
  1100   (12 em decimal)
+ 0110   ( 6 em decimal)
 ------
 10010   (18 em decimal)
```

Overflow

Signed

```
  1100   (-4 em decimal)
+ 0110   ( 6 em decimal)
 ------
 10010   ( 2 em decimal)
```

Não há Overflow

E em resumo...

***Overflow:** É quando o resultado de uma operação aritmética está fora do intervalo representável de inteiros.*

- Os números têm dígitos infinitos, mas os computadores têm precisão finita. Isso pode levar a erros aritméticos.
- **Atenção!** *Overflow* é identificado de formas diferentes quando estamos trabalhando com *unsigned* e *signed.*

Unsigned

```
  0100   ( 4 em decimal)
+ 0110   ( 6 em decimal)
 ------
  1010   (10 em decimal)
```

Não há Overflow

Signed

```
  0100   ( 4 em decimal)
+ 0110   ( 6 em decimal)
 ------
  1010   (-6 em decimal)
```

Overflow

Exercícios

Exercícios

1. Qual é o número mínimo de bits necessários para representar exclusivamente os caracteres do alfabeto português?

2. Dada a resposta anterior, quantos caracteres a mais poderiam ser representados sem a necessidade de bits adicionais?

3. Usando 7 bits para representar cada número, escreva as representações de 23 e -23 em sinal magnitude e em complemento de dois.

4. Qual é o maior número positivo que podemos representar em complemento de dois com 12 bits? Escreva seu resultado em binário e decimal.

5. Qual é número negativo de maior magnitude que pode ser representado com 12 bits em complemento de dois? Escreva seu resultado em binário e decimal.

6. Qual é o maior número positivo que se pode representar em complemento de 2 com n-bits?

7. Qual é o número negativo de maior magnitude que pode ser representado com n-bits em complemento de dois?

Exercícios

8. Converta os números representados em complemento de dois para decimal:

   a. 0110

   b. 1101

   c. 0110 1111

   d. 1101 1011 0001 1100

9. Expresse o valor negativo -22 como um inteiro de complemento de 2, usando 8 bits. Repita para 16 bits e 32 bits. Você percebe um padrão nas respostas?

10. Descreva quais condições indicam que ocorreu estouro (*overflow*) quando dois números em complemento de 2 são somados.

11. Os números binários a seguir são números binários de 4 bits representados em complemento de 2. Qual das seguintes operações gera *overflow*? Justifique suas respostas convertendo os operandos e os resultados para decimal.

    a) 0011 + 1100
    b) 0111 + 1111
    c) 1110 + 1000
    d) 0110 + 0010

Exercícios

Questão 1 _____ (*1.8 pontos*)

Complete a tabela a seguir com as representações equivalentes do número binário (em complemento de dois), hexadecimal, inteiro sem sinal (*unsigned*) e inteiro com sinal (*signed*). Siga o exemplo da primeira linha. Para receber o crédito da questão, **apresente todos os cálculos na folha de rascunho**.

Questão	Binário (complemento de dois)	Hexadecimal	Inteiro sem sinal (*unsigned*)	Inteiro com sinal (*signed*)
Exemplo	$1110\ 0010_2$	$E2_{16}$	226_{10}	-30_{10}
(a)	$1000\ 0010_2$	_____	_____	_____
(b)	_____	BB_{16}	_____	_____
(c)	_____	_____	12_{10}	_____
(d)	_____	_____	_____	-8_{10}